

Beyond Cryptographic Validity: Situation-Aware Security for Future Satellite Operations

Linan Huang, Houdong Wang, and Linling Kuang

Abstract—XXXX

I. Introduction

Future satellite systems are evolving from remotely commanded communication terminals into networked, multi-functional, and increasingly autonomous computing platforms. As information processing, computing, and decision-making capabilities increasingly migrate from the ground to onboard satellite systems, both the importance and the scope of onboard security are expanding accordingly. In addition to communication data, security mechanisms must now protect increasingly complex onboard states, processing logic, and control procedures.

Satellite communication systems have developed a broad range of cryptographic protection mechanisms across multiple protocol layers. Representative examples include the CCSDS Space Data Link Security Protocol [?] for conventional space links and Internet security protocols (e.g., IPsec) that can be adapted to emerging IP-based integrated space-terrestrial networks [?]. These mechanisms can authenticate message origins, ensure message integrity, and protect message confidentiality. However, cryptographic validity alone does not guarantee operational legitimacy. A message may be correctly authenticated and integrity-protected, yet still be inappropriate under the current operational context. For example, a telemetry, tracking, and command (TT&C) command originating from a compromised user terminal should neither be executed nor even forwarded to the onboard command and data handling computer, because a user terminal is not authorized to perform spacecraft control operations. Future satellite security must extend beyond the traditional CIA triad toward situation-aware protection that evaluates the operational legitimacy of authenticated actions and safeguards the correctness, continuity, and resilience of onboard operations.

Addressing this gap gives rise to three fundamental challenges. The first challenge lies in reconciling state complexity with trustworthiness. Future satellites need

to maintain rich and evolving security states to enable resilient secure mechanisms such as clear/protected mode management and transition [?]. However, greater state complexity makes the management logic harder to verify and expands its attack surface. The key question is therefore how to support complex security-state evolution while keeping the security-critical state-management core trustworthy and attack-resilient.

The second challenge concerns XXXX

The third challenge concerns system-level verification of operational legitimacy. In increasingly networked, distributed, and autonomous satellite systems, relevant evidence is often fragmented across nodes and operational stages, and may be heterogeneous, partial, or inconsistent. The key question is how to design mechanisms that actively trigger and orchestrate the systematic use of such evidence to derive reliable security knowledge and judgments, while adapting to satellite-specific architectures and stringent SWaP constraints.

This article aims to fill these gaps. To address the first challenge, we propose a minimally decoupled architecture that separates complex, mission-specific state management from a minimal and standardized cryptographic unit through well-defined control and data interfaces. On this basis, we define the responsibilities of the two units and establish a disciplined processing order for state evaluation and cryptographic execution, ensuring consistent handling of multiple interdependent security states.

For the second challenge of XXX, we XXX.

To address the third challenge, we adopt a role-aware strategy. Rather than burdening every node with equal security obligations—or expecting a single node to fully validate an operation alone—we distribute differentiated responsibilities among source, forwarding, destination, and execution nodes. Assignments are tailored to each node's place in the workflow, its available context, and its practical verification capacity.

As shown in Fig. 1, the overall structure of this paper constructs to expand traditional crypto validity to situation-aware security by solving three challenges in Section II, III, IV. The first can be viewed as extending conventional cryptographic processing to complex system states, thereby laying a trustworthy foundation. The second and third further advance toward situation-aware security from the perspectives of perceptibility and verifiability, respectively.

L. Huang is with the Beijing National Research Center for Information Science and Technology (BNRist), Tsinghua University, Beijing 100084, China (e-mail: huanglinan@tsinghua.edu.cn).

H. Wang is with the Department of Electronic Engineering, Tsinghua University, Beijing 100084, China (e-mail: hd-wang21@mails.tsinghua.edu.cn).

L. Kuang is with the Beijing National Research Center for Information Science and Technology, and the State Key Laboratory of Space Network and Communications, Tsinghua University, Beijing 100084, China (e-mail: kll@tsinghua.edu.cn).

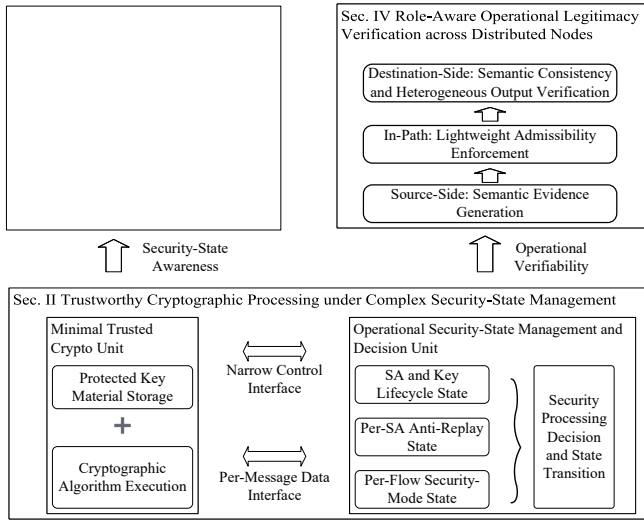


Fig. 1: XXXX

II. Trustworthy Cryptographic Processing under Complex Security-State Management

A. Security-State Complexity and Design Principles

The complexity of onboard security-state management is increasing along three closely coupled dimensions. First, modern satellites simultaneously carry heterogeneous traffic flows with different criticality and protection requirements, making per-flow security management preferable to a single link-wide configuration. Mission-critical flows such as telecommands can further support both protected and clear modes [?] to preserve essential functions in the event of onboard cryptographic failures. Second, each flow is associated with a corresponding Security Association (SA), while different SA may reference either distinct keys or a shared key. This creates nontrivial flow-SA-key dependency and mapping relationships, in which a change to one security object may affect multiple flows or processing contexts. Third, key management is evolving from the use of a statically preloaded key pool toward support for over-the-air key updates. This evolution introduces temporal complexity through key generation, activation, rollover, expiration, and revocation. It also introduces distributed-state consistency requirements, because the onboard and ground segments must maintain synchronized views of the active keys despite frame loss, transmission errors, delayed acknowledgments, and intermittent connectivity.

Consequently, system complexity increasingly stems not from the cryptographic algorithms themselves, but from the evolving security states that govern when, where, and how cryptographic operations are invoked. This expanded state-management logic also enlarges the attack surface: an adversary may corrupt a flow's clear/protected state, manipulate flow-to-key mappings, or disrupt key synchronization between the satellite and ground segments, causing legitimate traffic to be rejected or unauthorized operations to be accepted. These observations motivate the following two design principles.

Principle 1 (Keep the Cryptographic Core Small). Security-state management should be decoupled from cryptographic execution, leaving only key-dependent functions within a minimal and tightly controlled cryptographic boundary.

Principle 2 (Low-Cost Checks First, Reject Early). Security checks should proceed from simple, key-independent checks to costly cryptographic operations, enabling early rejection of invalid requests and reducing denial-of-service risks.

The following Subsections II-B and II-C present a decoupled architecture and a progressive security-processing order based on these principles.

B. Decoupled Architecture with Controlled Interfaces

CCSDS [?] provides a well-established precedent for the first principle by managing Security Associations (SAs) and cryptographic keys through separate but coordinated lifecycles. An SA transitions through Null, Unkeyed, Keyed, and Operational states, whereas a key independently transitions through Pre-Activation, Active, Deactivated, and Destroyed states. The two are associated through the Rekey SA procedure.

Building on this established separation, we broaden the managed state space beyond SA and key lifecycles. This broader state space requires the trusted boundary to be redrawn between operational state management and cryptographic execution. As illustrated at the bottom of Fig. 1, the proposed architecture comprises a Minimal Trusted Cryptographic (MTC) unit and an Operational Security-State Management and Decision (OSMD) unit. The MTC unit is limited to protected key-material storage and cryptographic algorithm execution, with its executable logic fixed before launch and not reconfigurable in orbit. The OSMD unit, by contrast, maintains all operational security states, including per-flow security-mode states, per-SA anti-replay states, and the lifecycle states of SAs and keys. It is also responsible for security-processing decisions, with reconfigurable logic that can be updated in orbit to accommodate evolving operational requirements and attack strategies.

All external requests terminate at the OSMD unit, leaving the MTC unit unexposed to other onboard or ground entities. The OSMD unit receives space-ground messages, validates them against the current operational security states, and determines how they should be processed under satellite-specific constraints. In particular, its processing order accounts for limited size, weight, and power (SWaP) resources and the resulting susceptibility to denial-of-service attacks, as discussed in Subsection II-C.

The OSMD and MTC units interact through two narrowly defined internal interfaces. The first is a control interface that supports only operations directly involving cryptographic key material. For a symmetric-key implementation, operational keys are derived within the MTC unit from preloaded root keys and are securely

deleted when no longer needed. All other key-lifecycle states and decisions, such as activation and deactivation, remain within the OSMD unit and are not perceived by the MTC unit. For example, deactivating a key only marks it as ineligible for subsequent use in the OSMD unit, whereas destroying the key additionally triggers its secure deletion from the MTC unit. Accordingly, the control interface carries only simple commands such as $(ParentKeyID, Derive, DerivedKeyID, Parameters)$ and $(KeyID, Delete)$, and returns only an execution status. No secret key material is transferred across this interface.

The second is a per-message data interface through which the OSMD unit requests cryptographic execution from the MTC unit. Each request contains the original message, the requested cryptographic service, the selected key identifier, and any operation-specific parameters. The MTC unit returns only the processed message and an execution status. Thus, the interface exposes cryptographic execution as a narrowly defined service while keeping all state interpretation and security decisions within the OSMD unit.

The above interface definitions provide a concrete symmetric-key implementation of the proposed architecture. Asymmetric cryptography can be supported by adding a small number of control operations, such as key-pair generation and shared-key establishment. The generated private key and shared secret remain within the MTC unit, while only public keys and execution status are returned to the OSMD unit. Therefore, supporting asymmetric cryptography does not alter the decoupled architecture or its minimal-interface principle; it only extends the set of cryptographic operations exposed through the controlled interface.

C. Dependency-Aware Security Processing

Consistent with Principle 2, Fig. 2 illustrates a progressive processing order across the OSMD and MTC units. The OSMD unit first performs lightweight, key-independent checks and invokes the MTC unit only after all applicable pre-cryptographic checks have succeeded. Persistent security states are modified only on the basis of cryptographically authenticated inputs. This ordering enables early rejection of invalid frames, conserves scarce onboard cryptographic resources, and prevents unauthenticated inputs from corrupting persistent security states.

The processing begins with per-flow protection-state validation. For each identified data flow, the OSMD unit maintains an authoritative System-Side Flow State (SFS), which specifies whether the flow is currently required to operate in clear or protected mode. Each received frame presents a corresponding Received Flow State (RFS) through its actual protection format. The OSMD unit proceeds only when the RFS is consistent with the SFS; otherwise, the frame is rejected before any SA processing or cryptographic operation is performed.

For a protected flow, each received frame carries a Received SA Identifier (RSAI), indicating the Security Association (SA) under which it claims to be protected. The

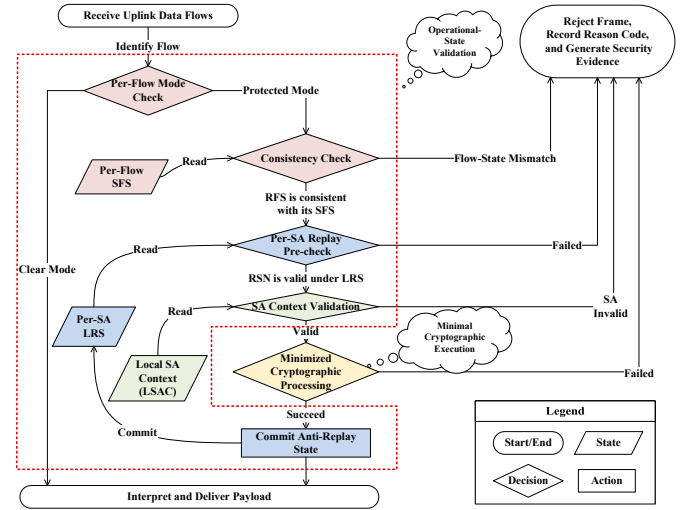


Fig. 2: Dependency-aware progressive processing across the OSMD and MTC units. The outer and inner red dashed boundaries delineate their respective processing scopes.

RSAI is treated only as an untrusted reference and is used to locate the corresponding Local SA Context (LSAC), which is authoritatively maintained by the OSMD unit. The LSAC determines whether the claimed SA and its associated cryptographic resources remain authorized for use. For example, an RSAI may reference an existing SA, while the key associated with that SA has already expired according to the locally maintained lifecycle state. In this case, the OSMD unit rejects the frame without invoking the MTC unit. Thus, the RSAI identifies the SA claimed by the incoming frame, whereas the LSAC determines whether that SA remains locally valid and operationally usable.

Once the RSAI has been resolved to an existing LSAC, the OSMD unit performs a lightweight anti-replay pre-check. We distinguish the Received Sequence Number (RSN), carried in the incoming frame, from the Local Replay State (LRS), which is maintained locally for the referenced SA.

CCSDS SDLS provides the baseline anti-replay mechanism. The receiver first verifies the integrity of a protected frame and then compares the authenticated RSN with the corresponding LRS. The LRS is updated only after both checks have succeeded [?]. This ordering prevents an unauthenticated RSN from modifying the persistent replay state. Otherwise, an attacker could inject a sufficiently large sequence number and cause subsequent legitimate frames to be rejected, resulting in a persistent denial-of-service condition. However, the baseline ordering also requires every received frame, including an evident replay, to consume cryptographic processing time and energy. Such overhead is particularly undesirable for onboard platforms subject to stringent size, weight, and power (SWaP) constraints.

To reduce this overhead, we introduce a two-stage replay

check inspired by IPsec ESP [?]. Before cryptographic verification, the OSMD unit compares the unauthenticated RSN with the current LRS. Frames that are evidently stale or lie outside the configured admissible advance range are rejected without invoking the MTC unit. Because the RSN has not yet been authenticated, this pre-check serves only as an early filter and cannot modify the LRS.

Frames passing the replay pre-check then undergo full LSAC validation. Only frames associated with a valid and authorized SA context are forwarded to the MTC unit for cryptographic processing. After successful cryptographic authentication, the OSMD unit performs the definitive replay check using the authenticated RSN and the current LRS. The LRS is updated to incorporate the RSN only if this final check succeeds. Consequently, the pre-check reduces unnecessary cryptographic operations, while the authenticated post-check remains the sole authority for committing replay-state transitions.

Overall, each processing stage relies only on information established by the preceding stages. The admissible frame set is progressively narrowed before cryptographic execution, while persistent state transitions are postponed until the corresponding inputs have been authenticated.

III. Constrained Security-State Perception and Reconstruction

Section II established how complex security states can be managed and applied onboard. A different question is whether the ground can obtain trustworthy and operationally useful knowledge of how those states evolve. The assurance target is an end-to-end service flow whose protection context may change with platform status, Security Association (SA) and key lifecycles, policy updates, routing changes, and satellite resets.

Consider a protected flow migrating from an old satellite path to a newly established path. Every participating node may individually report a healthy platform and an active SA, yet these local reports cannot establish whether the new protection context became ready before traffic migration or whether the old context remained valid until migration completed. The objective is therefore not to reproduce a complete copy of sensitive onboard states at the ground. Instead, the ground needs sufficient and trustworthy knowledge to determine whether selected flow-level security properties held over a specified operational interval.

This objective faces two distinct constraints. Some state values, cross-layer associations, and temporal relationships should remain within the onboard trust boundary, while intermittent and capacity-limited return links cannot carry all admissible evidence in real time. We therefore first define what the ground must distinguish, then transform the relevant state into disclosure-safe evidence, and finally select that evidence for progressive reconstruction.

A. Flow-Centric Security-State Semantics

Existing mechanisms provide separate views of individual security objects. CCSDS SDLS manages data-link SAs

[?], remote attestation reports platform integrity [?], and routing and policy functions maintain their own states. These views show whether a node is healthy, an SA is active, or a route is reachable, but not whether the objects jointly form the intended protection context of a service flow.

We use a persistent service-flow identity to relate five state categories at each participating node: platform and boot instance, protection mode and SA binding, key epoch, policy version, and route epoch. The identity remains stable when the path, SA, or key changes. The model also records security-relevant events such as SA establishment and withdrawal, rekeying, path migration, and reset recovery.

Four requirements determine whether the evolution is secure. State validity checks that each object remains usable. Binding consistency checks that the flow, SA, key, policy, and route epoch refer to the same protection context. Temporal continuity requires protection throughout the service interval, not only at reporting instants. Transition safety constrains the order of changes. For a path migration,

$$t_{\text{new protection}} \leq t_{\text{migration}} \leq t_{\text{old withdrawal}}. \quad (1)$$

The new context must be ready before carrying traffic, and the old context must remain usable until migration completes. After a reset, state and evidence must likewise bind to the new boot instance.

The output is a property-dependency model identifying the relationships, boundary events, and contexts needed to decide each flow-level property. It tells the next stage what must be demonstrated, without yet deciding how sensitive state is disclosed.

B. Disclosure-Safe Evidence and Sound Reconstruction

This stage temporarily assumes sufficient resources to deliver all evidence permitted by the disclosure policy. Its constraint is semantic: some information must not reach the ground verifier in raw form. An SA identifier, key epoch, policy, route change, or mission mode may reveal a sensitive value; their association and timing may further expose mission activities. Disclosure control must therefore cover values, cross-layer relationships, and temporal correlations.

The satellite therefore reports authenticated properties rather than raw state. Instead of disclosing a new SA and its activation time, it can assert that “the protection context associated with this flow and route epoch was operational before traffic migration.” The claim preserves the ordering required by (1) while hiding identifiers and, when required, exact event times.

Each claim binds the reporting node or role, a controlled flow identifier, boot and route contexts, validity interval, asserted property, provenance, and freshness. The OSMD unit evaluates the property against its authoritative operational state, while the MTC unit protects the claim-authentication key. This division keeps state

interpretation outside the minimal cryptographic core and prevents ordinary onboard software from forging evidence. Compromise of this authoritative evidence path is outside the assurance boundary.

The representation depends on sensitivity. A reportable property may use a signed assertion, a stable interval may use an authenticated summary, and a sensitive membership or relation may use a commitment or zero-knowledge proof. RATS provides the underlying attestation architecture [?], and constrained-disclosure attestation demonstrates property verification without complete measurements [?]. Our focus is the claim semantics that relate a persistent flow to its platform, protection, policy, route, and transition context.

Because disclosure hides part of the trajectory, the ground retains all evolutions consistent with the model and received claims rather than reconstructing one internal-state history. A property is Verified only when every remaining candidate satisfies it and Violated only when every candidate violates it. If safe and unsafe candidates remain, the result is Indeterminate; mutually inconsistent claims instead indicate an evidence or model anomaly. This conservative interpretation follows run-time verification under partial information [?]: hidden or missing information may reduce decisiveness, but cannot support an otherwise unjustified conclusion.

C. Resource-Aware Progressive Reconstruction

This stage inherits the disclosure boundary and adds limits on bandwidth, contact duration, storage, computation, energy, evidence lifetime, and decision deadlines. It selects only among admissible claims and never reveals prohibited state to accelerate reconstruction. Delay-tolerant networking can deliver evidence across disrupted contacts [?], but it does not decide which evidence deserves a scarce contact opportunity.

Evidence value depends on its contribution to the current judgment. Many stable-state reports may change nothing, whereas one claim that the new SA was ready before migration may remove the last unsafe candidate. Priority is therefore given to evidence that establishes a violation, enables a conclusive verdict, covers a critical transition, is about to expire, or supports other claims. Size, generation cost, energy, and deadline distinguish evidence of otherwise similar value.

Selected evidence must remain independently verifiable. The onboard function therefore summarizes stable intervals, creates authenticated checkpoints, and issues separate claims for critical transitions, avoiding logs that can be validated only in full. CCSDS SDLS illustrates this staging: a compact report can signal a security event, while detailed diagnosis may require later state or log retrieval [?]. Our follow-up is property-driven rather than an undifferentiated log request.

Reconstruction then becomes a closed loop. After each contact, the ground verifies claim provenance, context, interval, freshness, and model version before removing incompatible evolutions. Evidence awaiting bandwidth can

arrive later, policy-hidden evidence cannot be requested, and reset-invalidated or expired evidence requires a new claim. If the verdict remains Indeterminate, the ground requests only the unresolved property for the relevant flow, node or role, epoch, and interval.

Suppose the first contact reports migration and later withdrawal of the old context, but omits one new-path readiness claim. Both a safe evolution and an unprotected gap remain possible, so the verdict stays Indeterminate. The ground requests that readiness property for the affected node and route epoch. A later claim either establishes (1) or proves that migration preceded readiness. Resource scarcity may delay a conclusion, but cannot lower the threshold for Verified or Violated. The resulting record—flow, property, verdict, contexts, and validity interval—provides Section IV with bounded security knowledge for operational-legitimacy verification.

IV. Role-Aware Distributed Verification of Operational Legitimacy

The preceding two sections establish a trustworthy cryptographic foundation under complex security-state management and enable the ground segment to reliably perceive onboard security states, respectively. Building on this foundation, this section takes the next step toward operational legitimacy through system-level verification that actively generates and coordinates evidence across the system.

In future networked satellite systems with onboard routing and processing, a security-critical operation may traverse multiple nodes and processing stages before being ultimately executed. As illustrated in Fig. 3, we abstract this process into three node roles: source, transit, and destination. Because these roles have access to different information and support different defensive actions, we develop three complementary verification mechanisms tailored to their respective capabilities in Subsections IV-A, IV-B, and IV-C.

A. Semantic-Redundancy-Based End-to-End Verification

The first mechanism is motivated by the insight that the source node can determine the expected effect of a command when it is correctly executed by the destination node.

As indicated by the dashed lines in the green block of Fig. 3, conventional end-to-end verification of command effects is largely ground-centric and retrospective, typically relying on telemetry returned after onboard execution [?]. The destination node first executes the action carried in the uplink frame, after which the resulting state change is reported through telemetry. The source node can therefore complete the verification only after receiving the corresponding telemetry, potentially introducing substantial delay when constrained space-to-ground bandwidth permits the relevant state to be downlinked only at a low cadence.

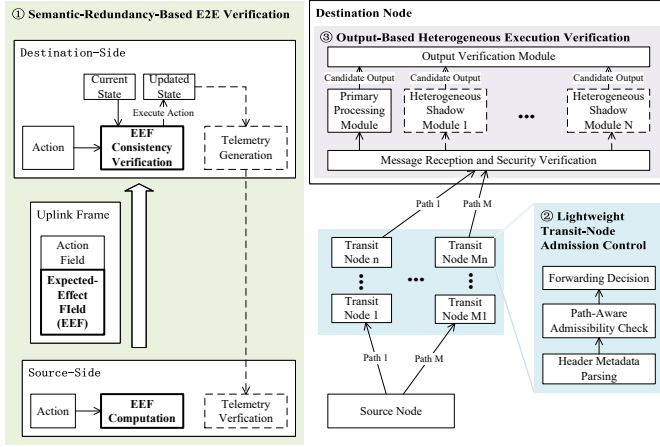


Fig. 3: Role-aware distributed verification of operational legitimacy across the source, transit, and destination nodes along the message-processing chain. The green, blue, and purple blocks represent three complementary mechanisms for verifying action-effect consistency between the endpoints, path admissibility at transit nodes, and execution consistency at the destination node, respectively.

To this end, we add an Expected Effect Field (EEF) to the uplink frame as a “semantic checksum” for pre-execution verification. As highlighted by the bold-outlined components in the green block of Fig. 3, the source node derives the EEF from the intended action and its locally maintained state, then embeds it in the frame. Upon reception, the destination node independently recomputes the EEF using the received action and its current state. The action is executed only when the two EEF values match.

By introducing semantic redundancy, the EEF enables real-time end-to-end verification before onboard action execution. As illustrated in the following example, it can detect two classes of inconsistency: unexpected alterations to the onboard state and incorrectly encoded actions, thereby preventing unintended execution.

Example 1 (Power-Level Adjustment). Consider an onboard transmitter with discrete power levels and two actions that increase or decrease the current level by one.

If the source node believes that the current level is k and issues an increase action, it sets the EEF to $k + 1$. The destination node independently derives the expected post-action level from the received action and its actual state, executing the action only if the result matches the EEF.

A mismatch therefore exposes either an unexpected change in the onboard state (e.g., malicious alteration or a reboot-induced reset) or an incorrectly encoded action (e.g., a decrease in place of the intended increase). In either case, the action is rejected before execution.

B. Lightweight Transit-Node Admission Control

Transit nodes typically inspect only the header fields required for forwarding. Given their SWaP constraints and

the high aggregate volume of multi-flow traffic, payload inspection or full-fledged IDS at every transit node is impractical and largely redundant with destination-side detection.

As shown by the blue block in Fig. 3, we instead enable transit nodes to perform lightweight admission control using compact, integrity-protected header metadata identifying the source, destination, and message type. Each transit node combines this metadata with its local path context to determine whether the traffic is admissible from its position in the forwarding path.

Consider the attack described in the introduction, in which a compromised user terminal injects a well-formed and cryptographically valid TT&C message. The first-hop onboard router observes that the message enters from the user-access domain and is classified as TT&C traffic. Because this domain is not authorized to originate such traffic, the router rejects the frame before forwarding.

Such early filtering is important because ingress provenance and other path-specific context may be lost or obscured after forwarding. Detecting the same violation at downstream nodes may therefore require substantially more expensive payload inspection or intrusion detection. A small amount of additional header metadata thus enables low-cost suppression of inadmissible traffic close to its point of entry.

Existing mechanisms provide partial foundations for this design. IEEE 802.1Qci [?] establishes an engineering baseline for lightweight forwarding-plane enforcement through Per-Stream Filtering and Policing (PSFP), whereas our mechanism extends such enforcement from traffic conformance to operational admissibility. Similarly, RFC 3704 [?] specifies ingress filtering and unicast Reverse Path Forwarding (uRPF) to assess whether a packet arrives from a direction consistent with its claimed source address. Our mechanism goes beyond source plausibility by further verifying whether the claimed source is authorized to originate the claimed message type.

C. Output-Based Heterogeneous Execution Verification

At the destination node, where an operation is ultimately executed, additional computational resources can be deliberately introduced to strengthen the assurance of security-critical operations.

Unlike the end-to-end mechanism in Subsection IV-A, which primarily verifies the consistency of the states maintained by the source and destination nodes, the mechanism proposed here focuses on the consistency of execution outputs. Directly comparing all internal states is often impractical in a complex system because the relevant states may be numerous, implementation-specific, or security-sensitive. In particular, some states, such as cryptographic keys or key-related identifiers, should remain confined within their respective hardware or trusted processing boundaries. We therefore verify execution consistency through externally observable outputs rather than through direct exposure or comparison of internal states.

As shown in the purple block of Fig. 3, the primary processing module is supplemented with one or more heterogeneous shadow modules. An incoming message first passes through the message authentication module, which performs the cryptographic checks described in Section II and the end-to-end semantic verification introduced in Subsection IV-A. Only messages that pass these checks are dispatched simultaneously to the primary processing module and the shadow modules. Each module processes the same authenticated input independently, and their outputs are forwarded to an output verification module. The operation is accepted only when the resulting outputs satisfy a predefined consistency rule; otherwise, execution is blocked, deferred, or reported as anomalous.

The use of heterogeneous implementations is essential to reducing common-mode compromise. Although redundant or backup processing units are widely employed in satellite systems, homogeneous replicas may share the same hardware vulnerabilities, software defects, or exploitable implementation assumptions. We therefore advocate implementing the shadow modules using diverse hardware architectures, software stacks, or processing paradigms, so that a single attack or fault is less likely to induce the same erroneous output across all modules. For example, the primary and shadow modules may employ different CPU architectures, while an additional shadow implementation may be realized on an FPGA. Such diversity allows output agreement to provide stronger evidence of correct execution than conventional homogeneous redundancy.

V. Case Study

attacks with wrong clear mode
attack with

VI. Conclusion

1. a decoupling structure instructed by principles. 2.
XXX 3. a systematic solution that let every component
at different position to contribute their own effect.